

Quick Sort Algorithm

01 April 2026 09:58

```
void quickSort(int arr[], int start, int end){
    if( start >= end){
        return;
    }

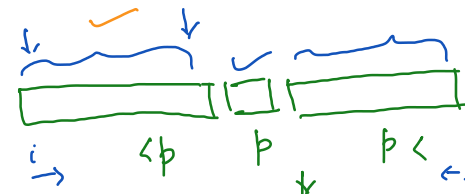
```

o/w
 $\nwarrow$
 int $\nwarrow$ p = partition(arr, start, end);

/* Sort the left part */
 $\therefore$ quickSort(arr, start, p-1);

/* Sort the right part */
 quickSort(arr, p+1, end);

}



← pivot has placed at its right place
 and elements smaller than pivot
 has been placed towards left and
 elements greater than pivot placed
 towards right.

```
int partition(int arr[], int start, int end){
```

int pivot = arr[start]; /* choosing start or first element as pivot */

int i; int count = 0;
 for(i = start+1; i <= end; i++){

if(arr[i] < pivot){

count++;

}

}

int pivotIndex = start + count;

swap(arr[start], arr[pivotIndex]);

$$O(n) + O(n) \\ = O(n)$$

```

i = start;
int j = end;
while( i < pivotIndex if j > pivotIndex) { ← O(n)
    while( arr[i] < arr[pivotIndex] ) { ← pivot
        i++;
    }
    while( arr[j] > arr[pivotIndex] ) { ← If this condition got false
        j--;
    }
    swap(arr[i], arr[j]);
    i++;
    j--;
}

return pivotIndex;
}

```

← if this condition got false than present element (arr[i]) is greater than pivot.

← If this condition got false than present element (arr[j]) is less than pivot.